

Flight Delay Prediction Using Open Transportation Data

Alina Iakubova

Computer Science and Engineering

Digital Transformation Management

University of Bologna

Email: alina.iakubova@studio.unibo.it

Abstract—This study develops a machine learning approach for predicting flight departure delays using historical airline, airport, weather, and operational data. The problem is formulated as a binary classification task, where a flight is classified as delayed if its departure delay is 15 minutes or more. The data preparation stage included missing value analysis, duplicate removal, type conversion, categorical encoding, feature scaling, outlier inspection, and feature engineering. Extreme but plausible observations, such as severe weather conditions, high traffic levels, or unusual airport activity, were retained because they may reflect real operational situations related to delays.

Several models were evaluated, including KNN, KNN with PCA, RIPPER, Gaussian Naive Bayes, Decision Tree, Random Forest, and LightGBM. The results show that flight delay prediction is a challenging task due to class imbalance and the limited ability of the available features to fully capture complex operational disruptions. Simpler models showed poor balance between precision and recall: RIPPER achieved near-zero recall, while KNN and Naive Bayes showed limited predictive power under class imbalance. The best result was achieved by the feature-engineered LightGBM model, which obtained an F1-score of 0.43 on the full test set. Feature importance and SHAP analysis indicated that departure hour, precipitation, and departure airport were among the most influential predictors. Overall, the study shows that machine learning can identify meaningful delay-related patterns, but accurate flight delay prediction remains difficult without richer real-time operational data.

1. Introduction

Flight delay prediction is an important problem for the aviation industry because delays affect airlines, airports, passengers, and the overall efficiency of air transportation. Even a single delayed departure can create additional costs, disrupt passenger connections, reduce aircraft and crew availability, and contribute to congestion across the network. For passengers, delays often mean missed connections, longer waiting times, and lower service satisfaction. For airlines and airports, early identification of flights with a high delay risk can support better resource planning, operational decisions, and disruption management.

This machine learning project is dedicated to predicting whether a flight departure will be delayed by 15 minutes or more. The task is treated as a binary classification problem, where the target variable indicates whether a flight is delayed or on time. The project uses historical flight, airline, airport, weather, and operational data to explore which factors are associated with departure delays and to compare the performance of different machine learning models.

The implementation is organized into several main parts:

- 1) **Data Import.** This stage loads the dataset from Kaggle and prepares it for further analysis. The project uses one CSV file, *full_data_flightdelay.csv*, which already contains both the target variable and the explanatory features. The target variable is `DEP_DEL15`, indicating whether a flight departure was delayed by 15 minutes or more. The dataset also includes flight, airport, airline, weather, and operational features, such as `DEPARTING_AIRPORT`, `CARRIER_NAME`.
- 2) **Data Understanding.** This section explores the structure of the data, descriptive statistics, target distribution, categorical feature distributions, delay rates, and relationships between delays and variables such as departure hour, month, airport, carrier, and weather conditions. This stage helps identify initial patterns and gives context for later modeling decisions.
- 3) **Data Quality Analysis.** This section checks whether the dataset is reliable enough for modeling and identifies which types of transformations are needed before training machine learning models. This stage includes an analysis of missing values, duplicates, data types, negative values, suspicious zeros, and logical consistency across variables. For example, weather variables are checked for plausibility: snowfall values are compared with months and airport locations to confirm that snow appears in reasonable seasonal and geographic contexts. The main purpose of this stage is to make sure that the data can be trusted, to detect potential data quality issues.
- 4) **Data Preparation.** This stage includes outlier detection and cardinality analysis. Outliers are not au-

tomatically removed, because extreme values may represent meaningful real-world conditions, such as severe weather or unusually high airport traffic. Instead, Isolation Forest is used as a diagnostic method, and an additional outlier flag is added as a feature for modeling. A cardinality check is also performed because the dataset is large and contains high-cardinality categorical variables, especially airport-related features. The relationship between features and the target variable is also analyzed using correlation and mutual information.

- 5) **Data Preparation and Modeling.** This section defines the feature set, target variable, train-test split, preprocessing logic, and model pipelines. Pipelines are used because they make it easier to combine preprocessing and modeling in a consistent way. Different models require different preprocessing strategies: for example, KNN needs scaling because it is distance-based, while tree-based models can work directly with unscaled numerical features. Categorical variables are handled using a combination of one-hot encoding and target encoding, depending on their cardinality and model requirements.
- 6) **Evaluation of Models.** This section compares model performance using classification metrics such as accuracy, precision, recall, and F1-score. Since the target classes are imbalanced, accuracy alone is not sufficient, so recall, precision, F1-score, and ROC-AUC are especially important for evaluating how well the models identify delayed flights. ROC curves are used to compare the models' ability to separate delayed and on-time flights across different classification thresholds.
- 7) **Model Interpretation.** For tree-based models, feature importance and SHAP values are used to interpret model behavior. These methods help identify which variables contribute most to the predictions and make the results more explainable. In this project, features such as departure time, precipitation, and departing airport appear to be among the most influential predictors of flight delay.

2. Proposed Methods

2.1. Data Understanding

This section focuses on exploring the structure of the dataset and identifying initial patterns related to departure delays. For better readability and interpretation, several variables are transformed into more understandable formats: months are represented in a clear 1–12 format, days of the week are converted into readable day labels, the departure hour is extracted from the departure time, and temperature values are converted from Fahrenheit to Celsius.

The first step is to analyze the target distribution. The dataset contains about 81% on-time flights and 19% delayed flights, which means that the classes are imbalanced. This

is an important finding because it affects model training and evaluation: accuracy alone may be misleading, so metrics such as precision, recall, F1-score, and ROC-AUC are needed later.

Next, bar charts are built for categorical variables together with delay rate overlays. This makes it possible to compare not only the number of flights in each category, but also the share of delayed flights. At this stage, a clear relationship between departure time and delay rate is observed: flights departing later in the day tend to have a higher delay rate. This leads to the hypothesis that departure hour should be an important predictive feature in the machine learning models.

Boxplots are then used to compare the distributions of numerical features for on-time and delayed flights. The goal is to understand whether delayed flights differ from non-delayed flights in terms of weather, traffic, aircraft, or route-related variables. The plots show that most numerical features have similar distributions for both classes. The most visible difference appears in `DEP_HOUR`, where delayed flights are more often associated with later departure times. Smaller differences are also observed in weather-related variables such as `PRCP`, `SNOW`, and `AWND`.

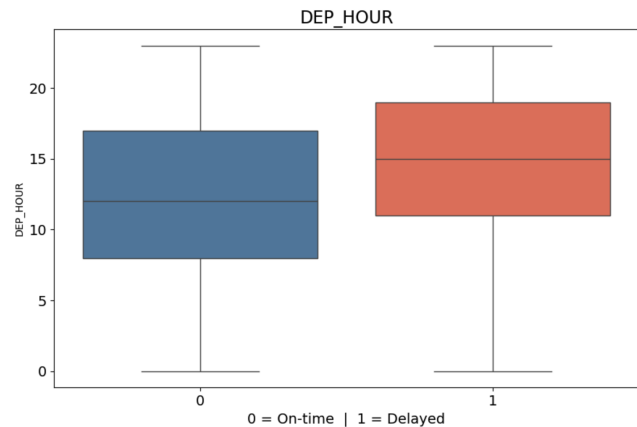


Figure 1. Feature distribution: Departure Hour

For `SNOW`, both delayed and on-time flights are mostly concentrated around zero, meaning that most flights took place without snowfall. However, delayed flights show a slightly heavier upper tail, suggesting that snowfall events are rare but may be more connected with delays when they occur.

2.2. Data Quality Analysis

The Data Quality Analysis section checks whether the dataset is reliable enough for modeling and whether any cleaning or transformations are required before training machine learning models.

First, missing values were examined. The dataset did not contain any NaN values. However, there were records where `PREVIOUS_AIRPORT` had the string value `NONE`. These records were analyzed separately to check whether an

empty previous airport always corresponds to the first flight in the aircraft sequence. This check was important because NONE in this column can represent meaningful operational information rather than a missing value.

Next, several data sanity checks were performed. Numerical columns were checked for impossible negative values, except for temperature-related variables, where values below zero can be valid depending on the measurement scale and weather conditions. Suspicious zero values were also reviewed, especially in weather columns.

The SNOW variable was analyzed in more detail to verify whether snowfall appeared in reasonable months and locations. Snow was not observed in summer months, which supports the reliability of this feature. Some snowfall records appeared in April and May. May snowfall records were all linked to Denver International (Stapleton), where late spring snow is common. April snow events were distributed across several airports at mid-to-high latitudes and were treated as plausible given the geographic diversity of US airports.

Temperature values were also reviewed by month. The distribution followed the expected seasonal pattern, with the highest maximum temperatures appearing during summer months. This supported the assumption that the weather data was generally consistent and meaningful.

Overall, no clear data anomalies were found in the dataset. In this context, an anomaly would mean an impossible or highly unrealistic value, such as an extreme temperature of 100 degrees Celsius, negative passenger or flight counts, or snowfall in an impossible seasonal and geographic context. The checks showed that unusual values mostly represented plausible real-world conditions, so the dataset was considered trustworthy for further preparation and modeling.

2.3. Data Preparation

The section focuses on preparing the dataset for modeling and deciding which transformations are appropriate for different algorithms.

First, outliers were inspected. In a flight delay dataset, outliers are not unusual: extreme values may reflect real operational situations, such as severe weather, very high airport traffic, unusual route characteristics, or exceptional aircraft activity. Because tree-based models are generally robust to outliers, these observations were not removed from the main dataset. For KNN, which is distance-based and therefore more sensitive to scale and extreme values, feature scaling was planned and applied later in the modeling pipeline.

Isolation Forest was used as an additional diagnostic method for outlier detection. The contamination threshold was set to 5%, meaning that the algorithm flagged approximately 5% of observations as potential outliers. The anomaly score distribution showed a relatively narrow left tail of more anomalous observations, while most records were concentrated in the normal range. This suggests that the majority of the dataset is consistent, while the detected outliers represent a small group of unusual cases rather than a broad data quality problem. Instead of deleting these

records, an outlier flag was later added as an additional feature.

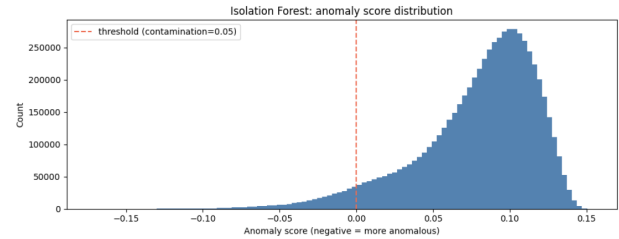


Figure 2. Anomaly score distribution

Next, feature cardinality was checked. This step was important because the dataset is large and contains categorical variables with many unique values, especially airport-related columns. High-cardinality features can become a bottleneck for one-hot encoding, creating too many columns and increasing memory usage and processing time. Therefore, different encoding strategies were considered for low- and high-cardinality categorical variables.

The relationship between features and the target variable was then examined. Pearson correlation showed that most numerical features had no strong linear relationship with DEP_DEL15, or the relationship was very weak. Because the target patterns may be non-linear, mutual information was also calculated. The mutual information scores were low in absolute terms, but they were still useful for ranking features and identifying which variables carried relatively more predictive signal.

Feature binning was also performed. Time of day, plane age, and distance were grouped into categories to test whether these more interpretable grouped features could help the LightGBM model capture non-linear patterns more effectively. Additional engineered features, such as an indicator for the first flight, were also created.

Finally, duplicate or highly correlated features were removed to reduce redundancy and noise. For example, variables such as AIRPORT_FLIGHTS_MONTH and AIRLINE_AIRPORT_FLIGHTS_MONTH were strongly related, so keeping both could add unnecessary noise without providing much new information. Redundant date and time features were also removed, such as keeping DEP_HOUR instead of the original departure time block. This helped simplify the feature set before modeling.

2.4. Model-Specific Preprocessing and Modeling

The goal of this section is to build models that are robust enough to work with a large real-world dataset and can produce reasonable predictive performance within an acceptable computation time. Since the target classes are imbalanced, with delayed flights representing only about 19% of the data, the main focus is not only on accuracy, but also on recall, precision, F1-score, and ROC-AUC. **Recall** is especially important because the practical goal is to identify as many delayed flights as possible. A model with low recall

may look accurate overall, but it would miss most delayed flights, which makes it less useful for delay prediction.

To keep the modeling process consistent and easier to manage, preprocessing and model fitting were organized using pipelines. This allowed each model to have its own preprocessing strategy while keeping the training and prediction workflow structured. For computationally expensive models, such as KNN and RIPPER, a sample of 100,000 training observations and 20,000 test observations was used. Faster and more scalable models, especially tree-based models, were later trained and evaluated on the full dataset. A fixed `random_state=42` was used throughout the project to make the results reproducible.

Model performance was evaluated using accuracy, precision, recall, F1-score, and ROC-AUC. Confusion matrices were analyzed to understand the types of classification errors, especially false negatives and false positives. ROC curves were plotted to compare how well the models separated delayed and on-time flights across different decision thresholds. For tree-based models, feature importance and SHAP values were also used to interpret which features contributed most to the predictions.

The first model tested was **K-Nearest Neighbors (KNN)**. Since KNN is a distance-based algorithm, numerical features were scaled using robust scaling, and categorical variables were encoded before training. Because KNN is computationally expensive on large datasets, it was trained on a sample. KNN was useful as a baseline, but it showed low recall and signs of overfitting, meaning it did not generalize well to unseen data.

A second KNN version was tested with **PCA**. PCA was added to reduce dimensionality and speed up distance-based learning. However, PCA did not improve the results. Since PCA preserves overall variance rather than target-specific information, it likely removed some predictive signal. Therefore, KNN with PCA was not selected as a final approach.

The **RIPPER** rule-based classifier was tested to see whether simple interpretable rules could identify delayed flights. The model produced very specific rules with high precision, but almost zero recall. This means it detected only a very small number of delayed flights and missed most positive cases, so it was not suitable for the main prediction task.

Gaussian Naive Bayes was used as a probabilistic baseline model. It required preprocessing of both numerical and categorical features. Since some categorical variables had high cardinality, target encoding was used as a more memory-efficient alternative to full one-hot encoding. GaussianNB performed better than KNN in terms of recall, but its assumptions were too simple for this dataset, so the overall F1-score remained limited.

The **Decision Tree** model was then tested because it can capture non-linear relationships and does not require feature scaling. It was first evaluated on a sample and then on the full dataset. The full-data version improved recall compared with simpler models and showed that tree-based methods are more suitable for this task. Feature importance and SHAP analysis were used to interpret the model, showing that

features such as departure hour, precipitation, and departure airport were especially influential.

The **Random Forest** model was trained on the full dataset as an ensemble of decision trees. It was expected to improve stability and reduce overfitting compared with a single tree. The model achieved relatively high recall, meaning it was better at detecting delayed flights, but precision remained low, indicating that many on-time flights were also classified as delayed.

Finally, **LightGBM** was tested as a more efficient gradient boosting model for large tabular data. It was trained on the full dataset and then improved with additional feature engineering. LightGBM achieved the best overall performance in terms of F1-score, showing the strongest balance between precision and recall. Feature importance and SHAP values were used to explain the final model and confirm that time, weather, and airport-related variables were among the most important predictors.

2.5. Results

The results show that flight delay prediction is a challenging **binary classification** task. During exploratory analysis, most numerical features showed strong overlap between on-time and delayed flights. Variables such as `CONCURRENT_FLIGHTS`, `NUMBER_OF_SEATS`, `PLANE_AGE`, and `DISTANCE_GROUP` did not clearly separate the two classes on their own. This suggests that individual features have limited predictive power and that the model needs to capture combinations of temporal, weather, airport, and operational factors.

The clearest pattern was observed in the **time-related features**. `DEP_HOUR` showed a visible difference between the two classes: delayed flights tended to have a higher median departure hour. This suggests that flights later in the day are more likely to be delayed, possibly because delays accumulate throughout the daily flight schedule. A similar pattern appeared for `SEGMENT_NUMBER`, where flights later in an aircraft's daily sequence were more often associated with delays. This supports the idea of **delay propagation**: if an aircraft experiences delays earlier in the day, later segments may also be affected.

Weather variables also showed relevant but less frequent signals. Features such as `PRCP`, `SNOW`, and `AWND` were highly skewed, with most observations close to zero. However, delayed flights showed slightly heavier tails for precipitation, snowfall, and wind speed. This indicates that severe weather conditions are not common in the dataset, but when they occur, they can contribute to higher delay risk. For this reason, weather-related outliers were kept in the dataset, as they may represent meaningful operational situations rather than data errors.

Mutual Information scores were relatively low in absolute terms, confirming that no single feature strongly explains the target variable by itself. However, the ranking was still useful: temporal and operational variables such as `DEP_HOUR`, `SEGMENT_NUMBER`, and airport-related features showed stronger dependency with the target than many

other numerical variables. This supports the need for models that can capture non-linear relationships and interactions between features.

Several **baseline models** were tested. **KNN** showed signs of overfitting, with substantially better performance on the training sample than on the test sample. It also required significant computational resources because it is a distance-based lazy learner: during prediction, each test observation must be compared with many training observations. **PCA** was tested as a dimensionality reduction step before **KNN**, but it reduced both train and test ROC-AUC, suggesting that the principal components did not preserve the most target-relevant information. Therefore, **KNN** was kept mainly as a baseline rather than as the main modeling approach.

Gaussian Naive Bayes performed differently. Although its accuracy was lower than **KNN**, it achieved better recall for the delayed class, meaning it was more able to detect delayed flights. However, its overall performance remained limited. On the full dataset, the combination of **TargetEncoder** and **GaussianNB** became problematic: **TargetEncoder** produced highly compressed, low-variance encoded features, and **GaussianNB** fitted overly narrow Gaussian distributions. As a result, the model tended to predict mostly the majority class, on-time flights. This made **GaussianNB** useful as a simple baseline, but not suitable as the final model.

The rule-based **RIPPER** model also struggled with the imbalanced target. Its accuracy appeared acceptable, but this was close to the majority-class baseline because most flights were on time. The model achieved very high precision for the delayed class but almost zero recall, meaning that it almost never predicted delays. This suggests that flight delays cannot be captured well by a small number of simple threshold-based rules.

Tree-based models were the most promising for this dataset. **Feature importance** and **SHAP analysis** showed that the model mainly relied on a small group of meaningful predictors: **DEP_HOUR**, **PRCP**, **DEPARTING_AIRPORT**, **SEGMENT_NUMBER**, and **PREVIOUS_AIRPORT**. Early departures generally reduced the predicted probability of delay, while later departures increased it. High precipitation values increased delay risk, and some departing airports were associated with historically higher delay probability. **SEGMENT_NUMBER** also had a positive effect on delay risk, again supporting the idea of accumulated delays during the day.

The table below summarizes the performance of all models evaluated in this project. For models tested on a sample, results are reported on the 20,000-observation test sample. For tree-based models, results are reported on the full test set of approximately 1.3 million observations.

3. Conclusion

This project addressed flight delay prediction as a **binary classification** task, with the goal of predicting whether a flight would be delayed by 15 minutes or more. The initial idea of predicting the exact delay duration in minutes was

	model	train_set	accuracy	precision	recall	f1	roc_auc
0	LightGBM - Feature Engineered	full (1.3M)	0.67	0.32	0.65	0.43	0.73
1	LightGBM	full (1.3M)	0.66	0.31	0.65	0.42	0.72
2	Random Forest	full (1.3M)	0.60	0.28	0.68	0.39	0.69
3	Decision Tree (full)	full (1.3M)	0.64	0.29	0.60	0.39	0.68
4	Decision Tree (sample)	sample (20k)	0.60	0.26	0.60	0.36	0.68
5	GaussianNB	NaN	0.73	0.29	0.28	0.29	0.62
6	GaussianNB	sample (20k)	0.73	0.28	0.27	0.28	0.62
7	KNN	sample (20k)	0.79	0.35	0.13	0.19	0.60
8	KNN + PCA	sample (20k)	0.78	0.25	0.08	0.12	0.52
9	RIPPER	sample (20k)	0.81	1.00	0.00	0.00	0.50

Figure 3. Models' evaluation results

not implemented because the selected dataset did not contain a numerical delay-duration target. Therefore, the final scope was adjusted to classification using **DEP_DEL15** as the target variable.

The analysis showed that flight delay prediction is a difficult problem. Most individual features did not clearly separate delayed and on-time flights, and the overall predictive performance remained moderate. This suggests that delays are influenced by a combination of factors rather than by one dominant variable. The most useful signals were related to **departure time**, **precipitation**, **departing airport**, and the aircraft's **segment number** during the day.

Among the tested models, simple baselines such as **KNN**, **Gaussian Naive Bayes**, and **RIPPER** had important limitations. **KNN** was computationally expensive and showed signs of overfitting, **GaussianNB** struggled with feature encoding and class imbalance, and **RIPPER** failed to detect most delayed flights. **Tree-based models** were more suitable for this tabular dataset because they could capture non-linear relationships and interactions between temporal, weather, airport, and operational features.

Overall, the project shows that the available features contain meaningful but incomplete information about flight delays. The best-performing model, **LightGBM with additional feature engineering**, achieved a ROC-AUC of 0.727 and an F1-score of 0.43. This indicates that the model performs clearly better than random guessing, but the separation between delayed and on-time flights is still challenging. A key limitation of this project is that the dataset mainly captures scheduled and historical operational attributes, while many real causes of delays, such as real-time air traffic conditions, crew availability, aircraft technical status, and previous flight disruptions, are not fully represented. Future work could include these real-time operational variables or use an alternative dataset containing exact delay duration in minutes for a regression task.

References

- [1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

- [3] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Alina Iakubova
May 6, 2026